Data Cleaning Approaches in Data Engineering Tasks with LLM Integrations

Jay Casey
Department of Software Engineering (Data Science)
Rochester Institute of Technology
Rochester, New York
jwc7817@rit.edu

Emily Nover
Department of Software Engineering (Data Science)
Rochester Institute of Technology
Rochester, New York
en7367@rit.edu

***Index Terms*—meaning-typed programming, byLLM, data cleaning, dirty datasets, prompt engineering, regex-based scripts**

I. Introduction

Software engineers now can reap benefits from the speed and productivity improvements of working with Large Language Models (LLMs). Programs recently gained the ability to directly implement artificial intelligence (AI) solutions into their code without needing to write thousands of lines to achieve the same result.

Prompt engineering is the current standard of this integration, and it involves connecting to a LLM (via APIs, websites, apps) and writing specific instructions in the code to explain exactly what should be done. This is a lengthy process and leaves much room for error when it comes to clarity within the codebase. As prompt engineering is considered as a newer topic, many engineers are still illiterate when it comes to speaking the LLM's language. ByLLM is a solution to this problem. By using Meaning-Typed Programming (MTP), byLLM completely removes the need to write these lengthy prompts. MTP uses minimal written code in order to interpret what functionality the user wants the LLM to contribute to the program [1].

However, this shift of responsibility from explicit prompt creation from the user to MTP also introduces another risk, misinterpretation. Since the user does not directly inform the LLM of the desired action, there might be preventable errors where the model's output does not match the intended action. This project investigates the accuracy, reliability, and maintenance overhead of using byLLM's MTP-driven approach for automated data cleaning compared to traditional prompt-based and regex-based approaches.

II. Problem Statement

ByLLM's MTP-driven structure simplifies the process of utilizing AI-assisted integrations in programming tasks down to simply using the by operator with a model of the user's choice. Overhead within tasks is reduced with byLLM by delegating a variety of tasks to an user-specified LLM provider such as Anthropic, Google, OpenAI, or a local model. The complexity of using AI integration in programming tasks is disguised, as code is generated based on the user's intent without the excessive prompting that comes with prompt-based programming.

There is a promising nature of MTP and byLLM, but an omission lies in the research on the capability of these tools to use the `by` operator to detect and repair dirty data automatically compared to traditional data cleaning methods such as regex-based scripting [1]. Traditional approaches in data quality tasks can be prone to error due to the naturally restrained nature of data cleaning methods to specific domains. Consideration is not given to the ability to adapt across domains, datasets, or prevent the influence of real-world noise in datasets. This is a case where the data cleaning pipeline does not produce output that matches the intended result (*e.g.,* incorrect values or inconsistent formats), and we define that as misinterpretation in this project.

MTP and byLLM appear to be potential solutions to tackling problems in data quality tasks that occur in traditional approaches, but there are challenges. MTP is found to be sensitive to poor coding practices [1], struggling with cryptic and bad namings. Many, not all, but many datasets are considered to be dirty due to cryptic and bad naming conventions, along with missing or corrupted data. These issues can appear across datasets in various domains (*e.g.,* government data, sports data, business intelligence data). ByLLM also relies mainly on application programming interfaces (APIs) and/or local-only models to access AI. As this project is not a funded project, API access (and by extension, the number of API calls) is limited to the free tiers of providers. Having limits on API access and calls will narrow down the amount of datasets usable for this project, reducing the full scope of the analysis.

Regardless, in this project, we will implement a data cleaning pipeline in three ways: *i)* byLLM with the `by` operator, *ii)* prompt-based scripts that use the same underlying LLM directly, and *iii)* traditional regex-based scripts. We will use subsets of real-world generic dirty datasets that have common data quality problems such as inconsistent date formats, missing values, and noisy categorical fields. Subsetting the datasets allows us to stay within the API limits while allowing for other people to reproduce this experiment to some semblance but with more computing power or a better API budget. Our goal is not to analyze the domain itself, but to evaluate how

byLLM, prompt-based LLM scripts, and regex-based scripts perform as engineering solutions for cleaning such datasets in terms of misinterpretation rate (*e.g.,* structure violations or incorrect repairs), development effort, and robustness across domains.

III. Related Work

Data cleaning has been a fundamental challenge in the data science world. Most real data encountered will require some level of preparation before it is ready to be used. Whether it be due to poor data collection methods, improper values, or corruption, the first step will always be cleaning. Previous work has focused on two specific stages of data cleaning: error detection and error repair. In many cases, the error detection and error repair stages are considered separate, and not viewed in relation to each other. This often shows results that are deemed bias. The bias comes from the metrics themselves being calculated on a proportional level and not on a quantity basis, allowing the opportunity to introduce new errors while showing high test scores [6].

Typical error repair does a single pass on the data to repair one type of error for each value. Some data cleaning models that utilize LLM's have implemented systems to iterate over the data, allowing the models to detect and repair multiple errors that may otherwise have remained unfixed [7]. Iterative models have concerns of hallucination, data accuracy, and overfitting. To address these concerns, a new step can be introduced to the iteration process of error validation [7]. This process is made to analyze any detected errors against the context of the dataset. Only after validation is complete can the repairer begin its process [7]. This flow relies heavily on the LLM's ability to interpret the data using similar awareness a human might have to notice the ambiguity and intention in the raw data.

Iterative frameworks emphasize the importance of splitting the data repair process into multiple stages: detection, validation, and repair [7]. Rather than treating iteration as a repeated process over the same task, these processes use the LLM in a structured sequence where each step influences the next. This design helps to reduce the amount of errors accross itterations and ensures that repairs are valid and correct for the context of the dataset. In contrast to single pass methods, this process captures more complex and multi-issue data errors while maintaining consistency and context within the data [7].

Other works have reviewed the difficulties of balancing automation with reliability in data cleaning models. While LLM based approaches provide strong, flexible abilities, they struggle with validity concerns due to inconsistent outputs and context misinterpretation. To address this, some methods incorporate constraint based reasoning and confidence scoring in order to filter and prioritize repairs [6]. This allows some control to be reclaimed by requiring the model to consider whether the suggested repairs will align with statistical patterns and logical rules found in the data source. This highlights the underlying pattern of dilemma between flexibility and control in modern LLM based cleaning models.

IV. Study Design

We will use Google Gemini via byLLM, direct API calls, and the webpage or app for Gemini. The rationale behind this decision lies in the fact that Rochester Institute of Technology (RIT) gives its student access to GenAI Tools [2] and Gemini is one of the two LLM on that list. Microsoft 365 Copilot is the second option on that list, but ease of access is an important aspect of this project, and Google Gemini becomes the winner due to students not having straightforward access to the Microsoft 365 suite compared to Google Gemini. Data cleaning using manual prompting will utilize the Gemini 3 model, as the current Gemini web/chat interface only allows the Gemini 3 model in Fast (free), Thinking (free), or Pro (paid) options. All byLLM-based cleaning will be used in the Gemini 2.5 family, which offers sufficient free-tier limits (see Section XII for details) compared to the Gemini 3 lineup for our sampled datasets. We estimate that we will make 250-500 [3] requests or less per day during the study and experiment phases of this project.

Approach Overview:

- Create subsets of dirty datasets to stay below the API limits.
- Data loading and basic preprocessing.
- Basic regex cleaning (date normalization, etc).
- Prompt-based LLM cleaning (hand-typed prompts, measure prompts, LOC, etc).
- byLLM cleaning (MTP types and `by` operator, measure LOC, misinterpretation, etc).
- Evaluation (compare outputs to a manually cleaned subset of datasets).

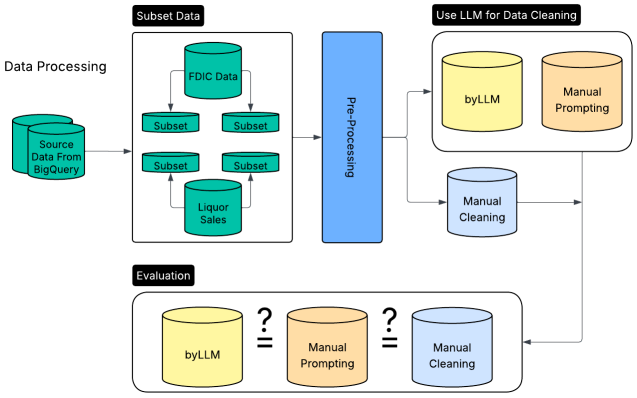

Fig. 1. Approach Overview Diagram.

As mentioned earlier in Section II, we define misinterpretation in this project as any instance where byLLM produces an output that violates the intended result (*e.g.,* wrong type, missing fields, or incorrect data) given a clearly defined prompt or specification. We will measure misinterpretation as the proportion of records with violations or incorrect repairs based on a ground-truth cleaned dataset.

As this project utilizes public LLM methods by the chat/web interface and the API access, we chose to provide the LLMs

with subset data from the datasets. This is to stimulate mitigating information leakage in real-world workflow scenarios. The data subsets provided to the LLMs were only 250 rows, and the LLMs were primarily used to generate the cleaning logic, which was then checked locally with the full dataset. This prevents the entire "dirty" dataset from being used with the model's training or memory.

We will use a sample from two "dirty" datasets:

- *Dataset 1*: FDIC-Insured Banks and Branches [4]
- *Dataset 2*: Iowa Liquor Retail Sales [5]

With these two datasets, we will follow the same steps and compare them to the "ground-truth" dataset. With the manual prompting (using Google Gemini's web/chat interface) and the byLLM, we will use prompts with different levels of specificity. With the web/chat interface, this prompt was used following the suggested guidelines (links can be found in the `manual_prompting.ipynb` file):

> *You are an expert software engineer and data scientist. Your task is to help me build a robust data cleaning and processing pipeline for a dataset that may contain inconsistencies, missing values, or formatting errors. I will provide a small sample of the dataset (250 rows) for demonstration purposes. Your pipeline should generalize to larger datasets without exposing the full dataset.*
> *Requirements:*
> *1. Data Ingestion*
> - *Suggest and write code to load a dataset (.csv, .json, or similar).*
> - *Handle potential read errors gracefully.*
>
> *2. Data Cleaning*
> - *Detect and repair dirty records, including but not limited to:*
> - *Inconsistent or invalid date formats*
> - *Missing or null values in any column*
> - *Misformatted numeric fields (extra characters, wrong decimal separators)*
> - *Misformatted categorical fields (typos, inconsistent capitalization)*
> - *Duplicate rows*
> - *Trailing or leading whitespace*
> - *Unexpected or invalid values (e.g., negative ages, impossible categories)*
> - *Outliers that are likely data entry errors*
> - *Any other records that might be considered dirty*
> - *Prioritize high precision (avoid incorrect corrections) and high recall (detect most errors).*
> - *Provide clear, step-by-step logic and code for each type of cleaning.*
>
> *3. Pipeline Skeleton*
> - *Structure the pipeline with modular functions (e.g., load_data(), clean_data(), analyze_data()).*
> - *Include docstrings and comments for maintainability.*
> - *Include a short demonstration of running the pipeline on the sample dataset.*
>
> *4. Validation & Testing*
> - *Suggest methods to check that cleaning worked correctly (e.g., counts of missing values, sample checks, consistency tests).*
> - *Optionally, provide small example inputs and expected outputs.*
>
> *5. Maintainability Notes*
> - *Explain why this approach is easy to maintain and extend.*
> - *Highlight potential bottlenecks or limitations.*
>
> *Output:*
> - *Python code implementing the pipeline, modular and well-documented.*
> - *Brief explanation of each module and function.*
> - *Demonstration of the pipeline applied to the sample dataset.*

Due to byLLM being built on MTP, a more simple, straightforward prompt was used:

> *Clean the data presented.*

To ensure that the experiment stays within free API tiers of Google Gemini, we will limit LLM-based cleaning to subset samples of each dataset if they exceed 250 rows and cache all responses [**?**] to avoid repeated calls. We will primarily use the Gemini 2.5 model, which falls in the free tier usage for APIs, for any byLLM-based cleaning, the Gemini 3 model for the web/chat interface for any manual prompting cleaning, and run manually-typed/regex scripts on the full datasets for ground-truth and baseline comparisions. This approach design allows us to compare the three ways while staying within rate limits and cost barriers.

V. Research Question

The research question for this project is primarily focused on the data quality abilities of MTP using byLLM.

RQ1 (Data Quality): For dirty datasets, can the `by` operator (via byLLM) detect and repair dirty records (*e.g.*, inconsistent date formats, missing labels) with lower misinterpretation rate and comparable or better precision/recall than regex-based scripts and prompt-based LLM calls?

VI. Experiments and Preliminary Results (FDIC dataset)

A. Precision, Recall, F1 Score

The manually typed cleaned dataset was used as ground truth to compare with the LLM approaches. This approach produced these column-averaged metrics (since there are 103 columns in the dataset) of the FDIC dataset:

- Average Precision: 0.9693
- Average Recall: 0.9671
- Average F1 Score: 0.9671

These results indicate that the manual prompting approach achieves a near-perfect alignment with the ground truth on a per-column basis. The average precision being 0.9693 means the manually prompted LLM produced close to a perfect match against the ground truth dataset. Incorrect values were rarely introduced, and the 0.9671 recall confirms that a large amount of the data that is expected is present. Unsurprisingly, the average F1 score is also very high at 0.9671, showing a balance between the manually typed and manually prompted cleaned datasets. This shows that the manual prompting method does succeed at delivering high-quality data cleaning without losing or hallucinating records, PROVIDED that there is a well-structured prompt and clear dataset structures are provided. Based on theses scores we can almost trust implicitly what the manual prompting produces.

byLLM comparision here

B. Misinterpretation Rate

For the manually prompted dataset, the column-level metrics performed very well, but the row-level metrics shows a more nuanced result. With the 250-row data subset, there were 249 matched rows, and 248 of these matched rows were labeled as having at least one column-level discrepancy between the manually prompted output and the ground truth. This yields a misinterpretation rate of 99.60%, and this appears to contradict the high precision and recall scores. However, this metric measures entirely fundamentally different things, as precision/recall evaluates the correctness per column in its own "area", while the misinterpretation rate flags any row where even a single column differs. As there are 103 columns being compared, even a very low per-column error rate will compound across columns because nearly every row will have at least one small discrepancy somewhere. It is important to emphasize here that row-level and column-level misinterpretation rates are not interchangeable metrics, but complementary ones. They can coexist but can not be compared with each other, and they both are necessary for analyzing the cleaning performance in this project.

byLLM comparision here

1) Column-Level Mismatches: To better understand where discrepanices happened in this dataset for manually prompted cleaning, a per-column mismatch rate was created with all 103 shared columns. The result was highly skewed:

- 76 columns had a 0% mismatch rate; matching perfectly to the ground truth.
- 94 columns had less than a 5% mismatch rate.
- 5 columns fell in the 5-25% mismatch rate.
- 4 columns had 25% or more mismatch.

The top mismatched columns were: `cbsa_division_fips_code` (77.1%), `change_code_1` (75.1%), `csa_fips_code` (36.9%), and `institution_name` (34.1%). The FIPS code columns are numeric identifier fields where the LLM might have reformatted values or dropped leading zeros. This is a simple error but could be a costly one if geographic work is needed in the future. `change_code_1` was mostly null in the 250-row subset, and it seems that the LLM handling of the nulls was different than the ground-truth. `institution_name` mismatches are attributed to the LLM's application of title case normalization rather than camel case originals in the ground truth. The best performing columns were all structured well, low-ambiguity, with consistent formatting in the raw data, which means that the LLM likely performs more reliably when the input data is already relatively clean and unambiguous.

byLLM comparision here

C. Structure Integrity

Comparison of structures between the manually prompted output against the ground truth showed the following differences:

- Dropped columns:
  - `change_code_2`
  - `change_code_3`
  - `change_code_4`
  - `end_effective_date`
- Invented columns: None
- Row count difference: -1

The manually prompted LLM did not create any new columns, which is positive. The three `change_code_N` columns dropped were majority null in the sample given to the LLM, so it seems that the model interpreted near-empty columns as irrelevant and removed them. However, the dropped `end_effective_date` column is more consequential as this is the recording of the closing date of a "structural event" for a banking institution, as this could be important for time-related analyses. The one missing row can also be due to the manually prompted LLM's outlier removal logic (which is 5 std. dev. on total_assets). This results proves that LLM-based cleaning does need domain-aware review, especially for sparse but important columns.

byLLM comparision here

D. Processing Time Across Different Subset Sizes

TABLE I
PROCESSING TIMES (SECONDS) FOR THE FDIC DATASET ACROSS DIFFERENT APPROACHES

	Subset Size (# of Rows)			
Dataset	250 rows	500 rows	1,000 rows	Full Dataset
Manual Typing	0.49s	0.50s	0.48s	0.77s
Manual Prompting	0.42s	0.51s	0.46s	0.76s
byLLM	0.00s	0.00s	0.00s	0.00s

The processing speeds can be viewed at Table I.

byLLM comparision here

VII. EXPERIMENTS AND PRELIMINARY RESULTS (IOWA LIQUOR SALES DATASET)

A. Precision, Recall, F1 Score

The manually typed cleaned dataset was again used as ground truth to compare with the LLM approaches. This

approach produced these column-averaged metrics (since there are 24 columns in the dataset) of the Iowa Liquor Sales dataset:

- Average Precision: 0.9529
- Average Recall: 0.9529
- Average F1 Score: 0.9529

Results are slightly lower than the FDIC dataset but still remain strong, which confirms that the manual prompting approach does generalize well across domains. The same precision and recall values indicate that when the LLM made errors, they were consistently balanced rather than being too much in one side of the dataset, creating a symmetrical error result unlike FDIC, where the dataset had a much larger column count and more varied mismatch distribution, which led to a slight divergence between precision (0.9693) and recall (0.9671).

B. Misinterpretation Rate

The row-level misinterpretation rate for the Iowa dataset was 100%, with all 250 rows containing at least 1 column-level discrepancy. Again, this is misleading when analyzed alone, but further investigation showed that the `Vendor Number` column was responsible for the mismatches across all 250 rows. 20 out of 22 columns showed zero mismatches, so it reinforces the finding that row-level misinterpretation rate is a poor standalone metric with analyzing LLM cleaning performance. One mishandled column will inflate the row-level rates to seem very bad while the majority of the data is still cleaned correctly.

1) Column-Level Mismatches:

- 20 columns had a 0% mismatch rate.
- 21 columns had less than a 5% mismatch rate.
- 0 columns fell in the 5-25% mismatch rate.
- 1 columns had 25% or more mismatch (`Vendor Number` with 100%).

The one problematic column is an ID field where the manual cleaning pipeline clearly cast values to a string and stripped any trailing .0 suffixes to preserve the leading zeros. The manually prompted LLM kept these values as floats, which made mismatches (e.g., 370 vs 370.0). THis is the same error type as seen with the FDIC dataset with the FIPS code columns, which suggests a pattern. This pattern: LLM-based cleaning pipelines do struggle with numeric ID fields where string representation is important.

C. Structure Integrity

The Iowa Liquor Sales structure comparision shows no dropped columns, no invented columns, and no row count differences, which is a perfect match with the structure. This is different from the FDIC dataset where 4 colulmns were dropped, and the cleaner outcome for the Iowa Liquor Sales is likely attributable to the simpler structure of the dataset. There was only 24 columns versus 121 in the original FDIC dataset, which gives the LLM less opportunity to misinterpret sparse/ambigious columns.

TABLE II
PROCESSING TIMES (SECONDS) FOR THE FDIC DATASET ACROSS DIFFERENT APPROACHES

	Subset Size (# of Rows)			
Dataset	250 rows	500 rows	1,000 rows	Full Dataset
Manual Typing	0.53s	0.54s	0.52s	238.63s
Manual Prompting	0.35s	0.40s	0.43s	483.30s
byLLM	0.00s	0.00s	0.00s	0.00s

D. Processing Time Across Different Subset Sizes

VIII. Dataset Comparisons

Both the FDIC and Iowa Liquor Sales results reveal consistent patterns across both datasets. Column-level precision is high in both cases, and row-level misinterpretation is also dominated by a few mishandled columns rather than widespread random errors. With both cases, the columns that caused problems were numeric ID or code fields, with formatting conventions (e.g., strings, zeros, decimals) might differ from the ground truth and the LLM output. The well-structured categorical/date/financial columns consistently achieved 0 or near-0 mismatch rates with both datasets as well, suggesting that manual prompting is a reliable cleaning approach for most column types. The predictable and identifiable failures were concentrated in the ID and code fields.

IX. Technical Challenges

We encountered some challenges in building this study. The biggest challenge was the lack of uniformity between model versions. The byLLM API integration only supported Gemini 2.5, while the web/chat interface used for manual prompting was limited only to the Gemini 3 model. This is a bit of an issue since each model version will have differences in architecture and capabilities, meaning fully comparable outputs across approaches might not happen. This introduces a potential bias in evaluating performance and misinterpretation rates, since the differences between model versions may be at fault rather than the experimental approach. Since this challenge could not be avoided due to budget limitations, we tried to standardize the prompts as closely as possible between the two experiments that utilize LLMs, documented everything (from prompts to outputs), and made sure we considered model mismatches in our interpretation of results.

Building the byLLM pipeline proved to be a difficult task. The library is optimized for primitive data types, custom classes, or objects, all of which are not well optimized for this level of data manipulation within python. This was eventually resolved by combining all three in a sequence to broaden the capabilities of the `by` operator. Another hurdle came in the form of output limits. As the process involved reading the entire data set, altering and returning all the data, the LLM quickly reaches a token limit for each request. This introduces the need for batching/caching in order to separate the data into digestible pieces.

With that in mind, we also had limitations with API access and rates. We are using only the free tier of Google Gemini, and it restricts the number of requests and token size, which in turn limits our dataset size and results. While using batching and caching, the strain on the API limits are highlighted, forcing multiple requests in quick succession. While the batching addresses the token limit, it causes the request limit to quickly hit the daily quota, hindering our ability to clean data and test the pipeline.

On the LLM side, we had to deal with potential model misinterpretation/semantic ambiguity, reproducibility between domains and datasets, and the dependence on a certain model type. Most of the current LLMs are black boxes with changing architectures and versions, which might cause different results after updates or small changes. This impacts reproducibility over time and changes the cleaning behaviors of the models used. To reduce this, we made sure to stay with specific API versions, logging data, and repetitions of prompts/outputs. One of the biggest issues with MTP and byLLM is their inability to infer on cryptic or bad naming conventions, and MTP's reduction of explicit prompting might cause the model to misinterpret the intent of the prompt and data, causing incorrect or incomplete data cleaning. We mitigated that by making sure to choose dirty datasets with clear naming conventions or descriptive variable names, and validation approaches to double-check outputs.

Another model misinterpretation issue is making sure misinterpretations are measured objectively without accidentally bringing in bias or skewing the output results. What is considered to be "wrong", or "dirty"? We made sure to use our manually typed and cleaned datasets as ground truth and to set up strict rules for what was a "violation" or a incorrect repair.

Lastly, despite the LLM tendency to vary in their outputs based on how the prompt or data is structured, we made sure to select datasets that were as generic as possible to ensure that we could create clear evaluation metrics. We also documented everything, and created clear organizational structure within the repository and the files so any changes can be documented and made across all files.

X. Threats to Validity

1) Threats to Conclusion Validity: To measure the relationship between this cleaning approach and the outcomes reliably, conclusion validity is necessary. There are several factors constraining the statistical power of our findings in Phase 3. This limitation is acknowledged and will be addressed in Phase 4, where evaluation will be conducted on larger dataset samples than just 250 rows. The current results are internally consistent, but the sample sizes are small, and that increases the risk that the differences that are observed between approaches are associated with variations in samples rather than actual performance differences. The row-level misinterpretation rate does contradict the column-level precision, and interpreting them on a glance rather than understanding it can lead to different and possibly incorrect conclusions about LLM reliability. To mitigate this, this project made sure to show and explain both metrics along with their differences and similarities. It is also important to acknowledge that the use of weighted averages for the precision/recall/F1 scores across columns actually treats them equally regardless of their importance in their domains, so a column might carry the same weight as a trivial one, which might not really reflect real-world issues or importance.

2) Threats to Internal Validity: Internal validity looks at the differences we observe that is caused by the approach rather than an external factor. The most significant threat for this project is the model version mismatch between approaches. The manual prompting approach used Gemini 3 via the web interface, while the byLLM approach used Gemini 2.5 via API. These two models are architecturally different with different capabilities, meaning the observed differences in output quality between the two LLM-based approaches may reflect model capability differences rather than differences in prompting methodology. This threat could not be eliminated unfortunately, due to budget constraints. We acknowledge that this is a potential source of bias in cross-approach comparisons. The ground-truth dataset was cleaned by a human, which introduces potential human bias and error, and decisions made during that process reflects only the authors' interpretation of what is correct cleaning behavior. A different human making different but valid decisions will produce a different ground truth, which then would change the outcomes that happen in this project.

3) Threats to Construct Validity: This section looks to see if this project measures what it claims to measure. Our definition of misinterpretation, where any cleaned record that is different from the ground truth, is strict. Hence, what might be legitimate cleaning decisions made by the LLMs that are different but not that much different from the ground truth, will be counted as misinterpretations. For example, the title case application by the LLM for the `institution_name` column is a valid cleaning decision, but since the ground-truth dataset did not do this, it comes up as a mismatch. Consistency with the ground-truth dataset is preferred here rather than absolute correctness, since MTP is claimed to match the user's intent. The consistency choice may overstate the error rate, but again, the user intent is the priority here. Lastly, the 250-row subsets provided to the LLM for the manual prompting task, may not be fully representative of the dataset's variety of dirty records. It is known that the LLM comes up with its logic for cleaning based on the patterns it sees in the sample, so if the sample underrepresents the types of dirty data, then the resulting outcome might not generalize well to the full dataset even if it does well with the subsetted data.

4) Threats to External Validity: To ensure that this project satisfies the external validity requirement, where the findings generalize beyond this specific experiment, we used two datasets that both are public datasets with relatively well-structured structures and descriptive names. As already pointed out in the byLLM article [1], MTP is sensitive to cryptic or poorly named variables, and our dataset selection avoided this

as much as possible to ensure fair evaluations. Hence, our results likely represent a best-case scenario for LLM-based cleaning and may not generalize to datasets with ambiguous structures, domain-specific jargon, or badly corrupted records. Both datasets also originate from a similar domain, which is US-based public administrative data, which limits diversity. Results might be different for private-sector data, unstructured data, or datasets that are not English-based. This study is also conducted by student researchers for a class rather than professional software engineers, which may affect the quailty and representativeness of the manual cleaning ground truth and prompt engineering choices made during this experiment.

XI. Conclusion

This study reviewed the effectiveness of three data cleaning approaches: regex based manual cleansing, web based prompt engineering using publicly available LLM's, and MTP via byLLM. The results highlight both the strengths and limitations of LLM driven data cleaning processes.

Prompt-based approaches demonstrated strong performance in terms of precision, recall, and overall accuracy with the ground truth when provided detailed instructions on the desired cleansing outcomes. These results suggest that LLM's are capable of digesting large amounts of data and producing a clean dataset usable for analysis with ease. However, this approach introduces the additional step of crafting a prompt detailed enough to assist the LLM in its tasking.

Regex-based methods were used as our ground truth due to their high reliability and efficient, well defined transformational aptitude. However, regex remains highly constrained by its lack of flexibility, creating the need for reliable methods with a large amount of versatility to be used on a variety of datasets.

The byLLM approach, implemented MTP in attempts to provide the same level of accuracy as regex based scripting without the constraints. MTP offers a significantly decreased code complexity by enabling the LLM to interpret the desired result earlier in the code, allowing the user to write less logic and give the model more freedom by introducing less constraints. Its high level abstraction greatly improves readability for humans, but introduces a risk that the model may misinterpret the desired results, potentially causing unreliable results.

Overall, the findings suggest there is no single optimal approach for our data cleaning scenarios. Instead, each option has a trade off in control, flexibility, and code complexity. Prompt based approaches offer high accuracy at the cost of complexity, regex methods provide reliability with narrow domains, and byLLM enables simplicity and speed with potential risks in correctness.

Future work would focus on expanding the testing to larger and more diverse data sources, reviewing alternate LLM models, and identifying problematic areas where the alternate methods stray from the ground truth. Further research is also needed in exploring the intersection between the tested methods. While each have their own strengths and weaknesses, when combined they may prove to be more robust.

References

[1] Dantanarayana, Jayanaka L. and Kang, Yiping and Sivasothynathan, Kugesan and Clarke, Christopher and Li, Baichuan and Kashmira, Savini and Flautner, Krisztian and Tang, Lingjia and Mars, Jason. 2025. MTP: A Meaning-Typed Language Abstraction for AI-Integrated Programming. Proc. ACM Program. Lang. 9, OOPSLA2, Article 314 (October 2025), 29 pages. https://doi.org/10.1145/3763092

[2] GenAI Tools Available at RIT. https://www.rit.edu/its/generative-ai

[3] Gemini Rate Limits. https://ai.google.dev/gemini-api/docs/rate-limits#gsc.tab=0&gsc.q=free%20rate%20limits&gsc.sort=

[4] FDIC-Insured Banks and Branches. https://banks.data.fdic.gov/bankfind-suite/bulkData/bulkDataDownload

[5] Iowa Liquor Retail Sales. https://data.iowa.gov/Sales-Distribution/Iowa-Liquor-Sales/m3tr-qhgy/data_preview

[6] Wei Ni, Xiaoye Miao, Xiangyu Zhao, Yangyang Wu, Shuwei Liang, and Jianwei Yin. 2024. Automatic Data Repair: Are We Ready to Deploy? Proc. VLDB Endow. 17, 10 (June 2024), 2617–2630. https://doi.org/10.14778/3675034.3675051

[7] Wei Ni, Kaihang Zhang, Xiaoye Miao, Xiangyu Zhao, Yangyang Wu, and Jianwei Yin. 2024. IterClean: An Iterative Data Cleaning Framework with Large Language Models. In Proceedings of the ACM Turing Award Celebration Conference - China 2024 (ACM-TURC '24). Association for Computing Machinery, New York, NY, USA, 100–105. https://doi.org/10.1145/3674399.3674436

XII. Appendix

A. Free Tier Rate Limits (Per Project)

- Gemini 2.5 Pro: 2–5 requests per minute (RPM) and 50–100 requests per day (RPD).
- Gemini 2.5 Flash: 10–15 RPM and 250–1,500 RPD.
- Gemini 2.5 Flash-Lite: 15 RPM and up to 1,000–1,500 RPD.
- Token Limit: All models share a limit of 250,000 tokens per minute (TPM).
- Context Window: The 1 million token context window is available on the free tier for compatible models.
- *(THE ABOVE NEEDS A LINK FOR CITATION OR REFERENCE REASONS)*